St. John College of Engineering and Management
Autonomous Institute
(A Christian Religious Minority Institution)
Approved by AICTE and DTE, Affiliated to University of Mumbai / MSBTE
DTE Code : 3218 AICTE Permanent ID : 1-4790201
NAAC Accredited with Grade 'A+', Three Programs NBA Accredited

PROJECT REPORT

SMART BUSINESS ANALYTICS DASHBOARD FOR SALES, INVENTORY, AND CUSTOMER INSIGHTS

Submitted in partial fulfilment of the requirements for the award of the degree of

Third Year
Department
Electronics and Computer Science Engineering

Submitted by
Aarya M Raut (EU1247058)
Sahel Biswas (EU1247002)

Academic Year
2026–2027

College
St. John College of Engineering & Management

Faculty Advisor
Ms Aishwarya Churi

St. John College of Engineering and Management
Autonomous Institute
(A Christian Religious Minority Institution)
Approved by AICTE and DTE, Affiliated to University of Mumbai / MSBTE
DTE Code : 3218 AICTE Permanent ID : 1-4790201
NAAC Accredited with Grade 'A+', Three Programs NBA Accredited

CERTIFICATE

CERTIFICATE

This is to certify that the project report entitled "Smart Business Analytics Dashboard for Sales, Inventory, and Customer Insights Using Advanced DBMS and Web Technologies" is a bona fide work proposed and prepared by Aarya M Raut (EU1247058) and Sahel Biswas (EU1247002), of the Department of Electronics and Computer Science Engineering, in partial fulfilment of the requirements for the Bachelor of Technology degree.

Verification

The report describes the problem, proposed design, database structure, implementation approach, and testing plan for a web-based business analytics application. The final implementation status and test evidence must be verified by the project guide before submission.

Signatures

Faculty Advisor: Ms Aishwarya Churi Head of Department: _____

Date: _____ Place: _____

DECLARATION AND ACKNOWLEDGEMENT

DECLARATION

I declare that this project report has been prepared for academic purposes and that sources used in preparing the background and technical design are listed in the References section. Any implementation screenshots, test results, and performance figures added to the final submission must represent work actually completed and must not be fabricated.

ACKNOWLEDGEMENT

I express my sincere gratitude to my Faculty Advisor Ms Aishwarya Churi and the faculty members of the Department of Electronics and Computer Science Engineering for their guidance and feedback. I also acknowledge the authors and maintainers of the technical documentation and research literature consulted while planning this project. Finally, I thank my family and peers for their encouragement during the planning and development process.

REPORT NOTE

This is a project-report draft. Verify the faculty advisor and college details, insert genuine screenshots from the completed application, and update the Results section after running the system. The proposed scope intentionally focuses on PHP, MySQL, HTML, CSS, JavaScript, Bootstrap, and Chart.js; predictive machine learning and sales forecasting are outside the current implementation scope.

ABSTRACT AND CONTENTS

ABSTRACT

Small businesses generate sales and inventory records but may lack a convenient way to convert these records into timely, reliable business insights. This project proposes a responsive web application that combines transactional data management with a compact business intelligence dashboard. The application will support

secure login for Admin and Manager roles, customer and product management, sales entry, stock tracking, sales reports, and visual summaries such as sales trends, revenue by category, top customers, best-selling products, and low-stock alerts. The proposed technology stack consists of HTML, CSS, Bootstrap, JavaScript, Chart.js, PHP, and MySQL. Advanced DBMS concepts—including normalization, joins, aggregate queries, views, stored procedures, triggers, indexes, and transactions—will be used to maintain consistency and produce analytical summaries. The scope is deliberately practical for an undergraduate capstone: it emphasizes descriptive analytics and database-backed reporting rather than machine learning or predictive forecasting. The report presents the problem context, literature survey, feasibility, planning, system design, implementation approach, testing strategy, deployment considerations, and future enhancements. Actual screenshots and measured test outcomes should be inserted after implementation.

KEYWORDS
Business Intelligence, Sales Analytics, Inventory Management, PHP, MySQL, Advanced DBMS, Chart.js, Decision Support

CONTENTS

Chapter 1 — Overview
Chapter 2 — Literature Survey & Proposed Work
Chapter 3 — Analysis and Planning
Chapter 4 — Design & Implementation
Chapter 5 — Results & Discussion
Chapter 6 — Conclusion
References

CHAPTER 1 — OVERVIEW

1.1 Introduction

Business intelligence (BI) is the process of converting operational data into information that supports decisions. A small retailer may record every sale but still find it difficult to answer questions such as which products sell most often, which categories contribute the most revenue, when sales rise or fall, and which products are close to running out. When information is spread across paper registers or disconnected spreadsheets, producing these answers can be slow and error-prone. The proposed Smart Business Analytics Dashboard provides a central web-based system for recording core business transactions and viewing summarized indicators in one place.

System concept

The application is designed as a browser-based system. Users enter customer, product, inventory, and sales information through forms. PHP validates requests and communicates with MySQL. SQL queries calculate summaries, and Chart.js displays the resulting datasets as charts. The dashboard does not replace managerial judgement; it gives users a more consistent view of the information needed to make decisions.

CHAPTER 1 — BACKGROUND & IMPORTANCE

1.2 Background

Many small businesses begin with simple records because they are inexpensive and familiar. As the number of products, customers, and transactions increases, manual processes become harder to maintain. A spreadsheet can calculate totals, but users may need to copy data between files, reconcile inconsistent product names, or rebuild reports every month. A relational database can keep related information organized and reduce duplication. A web interface makes the same records available through structured forms, while an analytics dashboard presents a summarized view.

1.3 Importance of the Project

The project has operational and educational value. Operationally, it aims to reduce repetitive report preparation, highlight stock shortages, and help users compare sales performance across time periods and product categories. From a technical perspective, it combines web development with database design, transaction handling, access control, and visualization. From a business analytics perspective, it encourages users to define measurable indicators, interpret trends carefully, and connect observations to practical actions. Research on small-business analytics suggests that dashboards can support data-driven decisions, although benefits depend on sound metrics and active managerial use [1], [2].

CHAPTER 1 — STAKEHOLDERS, OBJECTIVES & SCOPE

1.4 Perspective of Stakeholders and Customers

Business owner/Admin: needs a consolidated view of revenue, products, and inventory, along with control over user access. Manager: needs to record sales, maintain customer/product records within assigned permissions, and review reports. Sales staff (possible future role): need fast and accurate sales entry. Customers: are indirect stakeholders; better record keeping can support more accurate service and purchase-history lookup. Project guide and evaluator: need to see a working application and evidence of database concepts. The design therefore prioritizes data accuracy, understandable screens, and traceable calculations.

1.5 Objectives and Scope of the Project

Objectives: (1) implement role-aware login and session management; (2) create CRUD operations for customers, categories, and products; (3) record sales and sale items; (4) update stock consistently after a sale; (5) provide dashboard KPIs and date-filtered reports; (6) demonstrate normalization, joins, aggregate queries, views, stored procedures, triggers, indexes, and transactions; (7) create a responsive interface. In scope: descriptive analytics, low-stock alerts, best-selling products, category revenue, monthly summaries, top customers, and basic report export where feasible. Out of scope: machine learning, predictive forecasting, RFM segmentation, external BI tools, complex APIs, and AI-based recommendations.

1.6 Summary

This chapter introduced the business problem and the purpose of the proposed dashboard. The system connects daily business operations with descriptive analytics while remaining feasible for an undergraduate project. The next chapter reviews related concepts and explains the proposed solution.

CHAPTER 2 — LITERATURE SURVEY & PROPOSED WORK

2.1 Introduction

This chapter reviews literature and technical resources relevant to business intelligence dashboards, small-business analytics, relational databases, and secure web applications. The objective is not to reproduce an existing commercial BI suite, but to apply established ideas in a small, transparent application that can be implemented and evaluated within an academic project.

Literature themes

Prior work on business intelligence and decision support emphasizes the conversion of data into useful information through a process that includes data collection, analysis, presentation, and managerial interpretation [2]. A field experiment on an analytics tool for small online retailers found that access to dashboard-style analytics was associated with improved seller outcomes, while also highlighting the importance of actual use [1]. These findings motivate a practical dashboard, but they do not guarantee that every implementation will produce the same business impact.

CHAPTER 2 — LITERATURE SURVEY TABLE

2.2 Literature Survey Table

Table 2.1: Summary of related literature and technical references

No.	Source / focus	Main contribution	Relevance to this project
1	Bar-Gill, Brynjolfsson & Hak (2023), NBER working paper [1]	Studies analytics tools and data-driven decisions among small online retailers; reports positive average revenue effects in the studied setting.	Supports the value of accessible analytics while reminding us to evaluate real usage.
2	Agwuna et al. (2022), BI dashboards for SMEs [2]	Reviews dashboard use, metric governance, visual communication, and decision routines.	Encourages clear KPI definitions and actionable, not decorative, charts.
3	Power (2007), Decision Support Systems [3]	Explains the role of computer-based systems in supporting managerial decisions.	Frames the dashboard as decision support rather than an automatic decision-maker.
4	MySQL Reference Manual [4]	Documents SQL, constraints, indexes, transactions, stored routines, and security-related features.	Technical basis for database implementation.
5	PHP Manual [5]	Documents server-side programming and session handling.	Guides PHP request handling and session-based authentication.
6	OWASP Session Management Cheat Sheet [6]	Provides guidance for session identifiers and web-session protection.	Informs secure login and session handling.
7	Chart.js documentation [7]	Explains browser-based chart creation from data arrays.	Supports responsive visual analytics.

Survey conclusion

The reviewed sources converge on three principles: data must be trustworthy, metrics must be clearly defined, and visual summaries should be connected to decisions. The proposed system follows these principles through relational constraints, reusable SQL summaries, and a limited set of understandable charts.

CHAPTER 2 — PROBLEM DEFINITION & FEASIBILITY

2.3 Problem Definition

Small businesses may store sales and stock information without an integrated method to summarize it. As a result, owners and managers can struggle to identify fast-moving products, compare monthly sales, calculate category contribution, or detect low stock promptly. The problem addressed is the absence of a lightweight, database-backed application that combines transactional record management with clear descriptive analytics. The solution must avoid duplicate or inconsistent records, prevent invalid sales quantities, and make reports understandable to non-technical users.

2.4 Feasibility Study — Technical

Technical feasibility is high for a student prototype because the stack uses common technologies: HTML/CSS/Bootstrap for layout, JavaScript and Chart.js for charts, PHP for server-side logic, and MySQL for relational storage. XAMPP can provide a local Apache/PHP/MySQL development environment. The application requires a standard laptop and browser; no specialized hardware is needed. The main technical risks are incorrect stock updates, unsafe SQL handling, session vulnerabilities, and inconsistent report calculations. These risks can be reduced through prepared statements, validation, database constraints, transactions, and systematic testing.

CHAPTER 2 — FEASIBILITY & METHODOLOGY

Feasibility Study — Economic and Operational

Economic feasibility is favorable because the main software components are available without paid licenses for a student project. Costs are limited to existing computing resources, electricity, internet access for documentation, and optional hosting. Operational feasibility depends on simple forms, readable labels, and a workflow that matches common retail tasks. A short user guide and realistic sample data will help demonstrate the system.

Feasibility Study — Schedule and Legal/Ethical

The project can be divided into requirements, database design, interface development, backend integration, analytics, testing, and documentation. The schedule is achievable if advanced forecasting and unnecessary integrations remain excluded. Real customer information should not be used in demonstrations without permission; synthetic data is preferable. Passwords must be hashed, access should be role-based, and the application should be deployed only with appropriate security settings.

2.5 Methodology Used

An incremental development methodology is proposed. Phase 1: gather requirements and define KPIs. Phase 2: create an ER model and normalize the schema. Phase 3: implement authentication and CRUD screens. Phase 4: add sales transactions and stock control. Phase 5: implement SQL reports and dashboard charts. Phase 6: test, correct defects, and document results. Each increment should be reviewed before the next is added. The system will be evaluated using functional test cases, data-integrity checks, usability observations, and verification that each dashboard metric matches its underlying SQL query.

2.6 Summary

The literature supports a measured, user-centered dashboard rather than a collection of charts without reliable data. The proposed solution is technically and economically feasible when the scope remains focused. The next chapter translates this scope into a plan, deliverables, and risk controls.

CHAPTER 3 — ANALYSIS AND PLANNING

3.1 Introduction

Analysis and planning establish what the system must do, which information it must store, and how development will be organized. The key requirement is to keep operational records and analytical summaries consistent. A sale should be saved only when its line items are valid and sufficient stock is available. Dashboard totals should be calculated from recorded transactions, not manually entered values.

Functional Requirements

FR-01: users can log in and log out. FR-02: Admin can manage user accounts and roles. FR-03: authorized users can create, view, update, and deactivate customer records. FR-04: authorized users can manage categories and products. FR-05: the system records a sale with one or more sale items. FR-06: stock is reduced consistently after a completed sale. FR-07: the system displays total sales, transaction count, and products sold for a selected period. FR-08: users can view category revenue, top customers, best-selling products, monthly sales, and low-stock products. FR-09: forms show clear validation messages. FR-10: reports can be printed or exported where the chosen implementation supports it.

CHAPTER 3 — NON-FUNCTIONAL REQUIREMENTS & PLANNING

Non-functional Requirements

Usability: navigation and labels should be consistent. Responsiveness: layouts should adapt to desktop, tablet, and mobile widths. Security: passwords are hashed; prepared statements are used; sessions are checked on protected pages; roles restrict sensitive actions. Integrity: primary keys, foreign keys, unique constraints, and transactions protect data. Performance: indexes are added to columns commonly used for joins and filters, after reviewing query patterns. Maintainability: database access, validation, and presentation logic should be organized into understandable files. Reliability: errors should be logged safely without exposing database credentials to users.

3.2 Project Planning

A sample eight-week plan is shown below. Week 1: requirements, KPI definitions, and initial literature review. Week 2: ER diagram, normalization, and schema creation. Week 3: authentication, sessions, and role permissions. Week 4: customer/category/product CRUD. Week 5: sales entry, stock update, and transaction handling. Week 6: SQL reports, views, stored procedures, triggers, and indexes. Week 7: dashboard charts, responsive styling, and report export if feasible. Week 8: integration testing, bug fixes, screenshots, and final documentation. The schedule should be adjusted to match the actual academic calendar.

Deliverables and Risk Control

Deliverables include requirement specification, ER diagram, normalized schema, SQL scripts, application source code, test-case sheet, genuine screenshots, and final report. Key risks are scope creep, insufficient time for database integration, incorrect financial calculations, and missing test evidence. Controls include freezing the MVP scope, testing stock updates early, defining each KPI before coding, maintaining backups, and recording defects in a simple issue list.

CHAPTER 4 — DESIGN & IMPLEMENTATION

4.1 Components and Hardware

Hardware: a standard laptop or desktop with a modern web browser; network access is required only for documentation or remote deployment. No sensors, microcontrollers, or dedicated hardware are needed because

this is a software-only business information system. Software: Windows/Linux/macOS, XAMPP or equivalent Apache/PHP/MySQL environment, PHP 8 or later, MySQL, a code editor, browser developer tools, Bootstrap, JavaScript, and Chart.js. A production deployment would use a supported server environment, HTTPS, restricted database access, backups, and environment-specific configuration.

Software Component Responsibilities

Frontend: provides navigation, forms, tables, responsive layout, and charts. JavaScript/Fetch: requests filtered summaries and updates page content without a full reload where useful. PHP: checks sessions and roles, validates inputs, executes prepared SQL statements, and returns HTML or JSON. MySQL: stores users and business records, enforces relational integrity, and executes analytical queries. Chart.js: renders chart types such as line, bar, and doughnut charts using server-provided values.

CHAPTER 4 — BLOCK DIAGRAM & FLOWCHART

4.2 Block Diagram

Figure 4.1 shows the proposed layered architecture. The browser collects input and displays results. The PHP layer validates requests and applies business rules. The MySQL layer persists records and calculates summaries. This separation makes it easier to maintain the interface without duplicating business rules in multiple screens.

Figure 4.1 — System Architecture

4.3 Flowchart

The high-level flow begins with login. Valid credentials establish a session and open the dashboard; invalid credentials produce an error without granting access. From the dashboard, the user selects a module, performs an authorized action, and receives a success or validation message. Sales entry must validate quantities and stock before committing the sale. A failed transaction should roll back all related changes.

Figure 4.2 — Login and Navigation Flow

CHAPTER 4 — UML & DATABASE DESIGN

4.4 UML

The use-case view summarizes the main interactions of Admin and Manager roles. Admin is responsible for user administration and can access the full set of management features. Manager access is limited to the operations granted by the role policy. Exact permissions should be enforced on the server, not only by hiding buttons in the interface.

Figure 4.3 — High-level Use-case Diagram

Database Design and Normalization

The relational schema is organized around users, customers, categories, products, sales, and sale_items. A user can create many sales; a customer can have many sales; a category can contain many products; a sale can contain many sale items; and a product can appear in many sale items. Primary keys identify rows and foreign keys preserve relationships. The schema

should be normalized to Third Normal Form (3NF): each table describes one entity or relationship, non-key attributes depend on the key, and repeating groups are separated. Product details should not be repeated in every customer record, and customer contact details should not be copied into each sale.

Figure 4.4 — Simplified ER Diagram

CHAPTER 4 — DATABASE OBJECTS & GUI

Advanced DBMS Concepts

Joins combine sales, sale_items, products, categories, and customers to create reports. Aggregate functions such as SUM, COUNT, and AVG calculate revenue, transaction counts, and average sale value. Subqueries can identify products whose stock is below a configured threshold. Views can expose reusable summaries such as monthly sales. Stored procedures can encapsulate repetitive database operations. A trigger may reduce stock after sale-item insertion, but trigger design must avoid double-decrementing stock if the PHP application also performs the update. Choose one authoritative stock-update mechanism and test it carefully. Transactions ensure that a sale header, line items, and stock changes either all succeed or all roll back. Indexes on foreign keys and common date/category filter columns can improve query performance; they should be validated with query plans.

4.5 GUI Screenshot

The following is a layout guide, not a screenshot of a completed application. Replace it with genuine screenshots after implementation. Recommended screens: (1) Login; (2) Dashboard with KPI cards and charts; (3) Customer Management table/form; (4) Product and Inventory table with low-stock indicator; (5) Sales Entry form; (6) Sales Reports with date filters. Screenshots should show real database-backed sample data and be labelled as figures in the final report.

Suggested Dashboard Layout

Top navigation: application name, logged-in role, and logout. KPI cards: Total Revenue, Number of Sales, Units Sold, and Low-stock Items. Main area: monthly sales line chart and revenue-by-category chart. Lower area: top-selling products, recent sales, and low-stock table. Provide a date-range filter and clear empty-state messages when no records match the selected period.

CHAPTER 5 — RESULTS & DISCUSSION

5.1 Actual Results

At the report-drafting stage, no verified application execution evidence or test logs were provided. Therefore, this section describes the intended outputs and the evidence that must be recorded; it does not claim that the system has already passed testing. After implementation, insert screenshots of successful login, customer/product CRUD, sale entry, stock reduction, and dashboard reports. Record the test environment, database seed data, test date, and observed result.

Expected Functional Outcomes

When implemented correctly, the application should maintain customer and product records, record sales with multiple line items, update inventory without allowing negative stock, and display totals consistent with the underlying transactions. Date

filters should change the displayed KPIs and charts. Low-stock alerts should list products at or below their configured reorder threshold. A report should be traceable to the SQL query and data records used to calculate it.

Discussion

The system's main contribution is integration: routine record keeping and descriptive analysis share the same database. This reduces the need to maintain separate report spreadsheets. The quality of insights still depends on correct product prices, stock values, sale dates, and transaction entry. Charts can reveal patterns, but users must consider promotions, stock-outs, holidays, and other business context before drawing conclusions.

CHAPTER 5 — FUTURE SCOPE, TESTING & DEPLOYMENT

5.2 Future Scope

Possible extensions include downloadable CSV/Excel reports, PDF report generation, audit logs, supplier and purchase-order management, multi-branch support, configurable reorder levels, and improved dashboard filters. Customer purchase analysis can be expanded only after defining valid business rules and protecting customer privacy. Predictive forecasting or customer segmentation may be explored as a separate future phase after sufficient reliable historical data exists; these are not part of the present scope.

5.3 Testing

Testing should cover valid and invalid inputs, authorization, relational integrity, calculations, and rollback behavior. Unit-level checks should test validation functions and KPI calculations. Integration tests should verify PHP-to-MySQL operations. System tests should exercise complete workflows from login to reporting. Security checks should include direct access to protected pages, SQL injection attempts against form fields, session expiry, and role restrictions. Use synthetic data and reset the database between repeatable test runs.

Table 5.1 — Test Plan

ID	Test scenario	Expected result	Status to fill
T01	Login with valid credentials	Correct role opens dashboard	Pending execution
T02	Login with invalid password	Access denied; safe error shown	Pending execution
T03	Submit customer with missing required field	Validation prevents invalid record	Pending execution
T04	Create product with negative price/stock	Input rejected	Pending execution
T05	Record sale with sufficient stock	Sale saved; stock reduced once	Pending execution
T06	Record sale exceeding available stock	Sale rejected; stock unchanged	Pending execution
T07	Force failure during sale transaction	Partial changes rolled back	Pending execution
T08	Filter report by date range	Only matching sales are included	Pending execution
T09	Manager attempts restricted admin action	Server denies unauthorized action	Pending execution
T10	Low-stock product threshold reached	Product appears in alert list	Pending execution

CHAPTER 5 — DEPLOYMENT

5.4 Deployment

For local demonstration, install XAMPP or an equivalent environment, start Apache and MySQL, create the project database, import the schema and seed data, configure database credentials in a local configuration file, and

open the application through localhost. The database credentials file should not be exposed in a public repository. Before a public deployment, configure HTTPS, supported PHP and MySQL versions, secure session cookies, least-privilege database accounts, regular backups, error logging, and server-side access controls. Disable debug output in production.

Deployment Checklist

1. Create and verify database schema. 2. Add an initial Admin account through a safe setup process. 3. Confirm password hashing and prepared statements. 4. Check that role permissions are enforced on each protected endpoint. 5. Test transaction rollback and inventory consistency. 6. Confirm charts use real query results. 7. Test responsive layouts in multiple browsers and screen sizes. 8. Back up the database and document restore steps. 9. Remove sample credentials and test accounts before public use. 10. Capture final screenshots only after the deployed build is stable.

Limitations

The prototype is designed for a single business context and descriptive reporting. Its reliability depends on accurate data entry and correct configuration. It is not a replacement for accounting software, tax filing systems, or audited financial statements. The term "profit" should be used carefully: a simple sales-minus-recorded-cost calculation does not include every operating expense, tax, return, or adjustment unless those data are explicitly modelled.

CHAPTER 6 — CONCLUSION

Conclusion

The Smart Business Analytics Dashboard for Sales, Inventory, and Customer Insights proposes a practical way for small businesses to combine transaction management with descriptive business reporting. By connecting a responsive web interface to a normalized MySQL database through PHP, the system can centralize customer, product, sales, and inventory records. SQL queries and Chart.js visualizations can summarize sales trends, category revenue, top customers, best-selling products, and low-stock items.

Learning Outcomes

The project offers experience in requirements analysis, relational data modelling, 3NF normalization, SQL joins and aggregation, stored procedures, triggers, views, indexing, transactions, authentication, role-based access control, form validation, session management, responsive design, and data visualization. It also reinforces an important business analytics lesson: a dashboard is useful only when the measures are well-defined, data is trustworthy, and users interpret the results in context.

Final Statement

The project is intentionally scoped as an achievable undergraduate capstone. The next practical step is to implement the database and core workflows, test inventory consistency, connect reports to real queries, and replace the report's illustrative design figures with genuine screenshots and measured results. Predictive analytics may be considered later, but a reliable descriptive dashboard is the priority.

REFERENCES

References

[1] S. Bar-Gill, E. Brynjolfsson, and N. Hak, "Helping Small Businesses Become More Data-Driven: A Field Experiment on eBay," National Bureau of Economic Research, Working Paper 31089, Mar. 2023. Available: https://www.nber.org/papers/w31089

[2] L. U. Agwuna et al., "Business Intelligence Dashboards for Timely Decision Support in Small and Medium-Sized Enterprises," DME Journal of Management, vol. 3, no. 1, pp. 71–91, 2022. Available: https://www.dmejournals.com/index.php/DMEJM/article/view/674

[3] D. J. Power, "A Brief History of Decision Support Systems," DSSResources.com, 2007. (Use as general background on decision-support systems.)

[4] Oracle, "MySQL 8.4 Reference Manual." Available: https://dev.mysql.com/doc/refman/8.4/en/

[5] The PHP Group, "PHP Manual: Session Handling." Available: https://www.php.net/manual/en/book.session.php

[6] OWASP Foundation, "Session Management Cheat Sheet." Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[7] Chart.js Contributors, "Chart.js Documentation." Available: https://www.chartjs.org/docs/latest/

[8] Bootstrap Team, "Bootstrap Documentation." Available: https://getbootstrap.com/docs/

Accessed: 21 October 2026.

Submission reminder
Before submitting: verify the bibliography format required by your department, confirm all references are cited in the report, replace placeholders, add actual GUI screenshots, and change every "Pending execution" test status to the observed result only after testing.